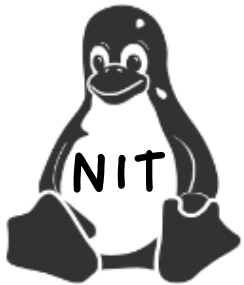


Welcome



Everyone



MASTERING LINUX!

Module 06

July 4th, 2026

SYSTEMD (Introduction)

RECAP OF SYSTEMD

Systemd is System and Service manager for Linux operating systems and provides tooling for controlling, reporting and system initialization

Systemd (Process Id- 1) is the first process which is **started during boot process** and starts other system resources.

Systemd manages the system through different objects ,in general known as **systemd units** which are representation of system resources and services.

Systemd unit consists of a name, type and configuration file that describes particular task. **systemctl utility** is used to manage the systemd units. Different types of systemd units (**#systemctl -t help**):

Service : Controls and manages different system services.

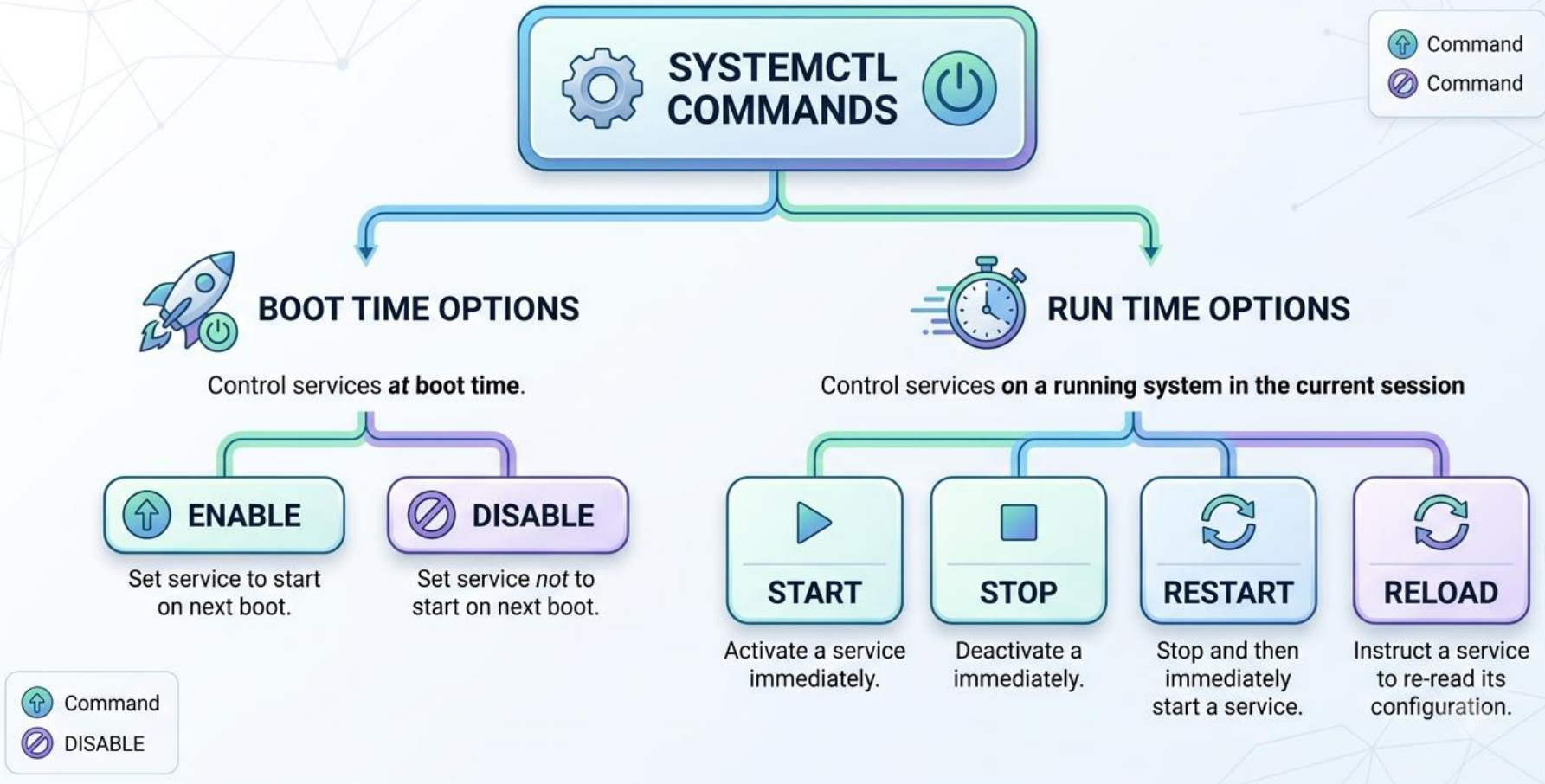
Target: Represents a group of units for specific system state.

Device : Manages hardware devices

Mount: Handles file system mounting

Timer: Manages task scheduling

SYSTEMD FILES (INFORMATION ONLY)



Categorization of systemd service control.

/var



#ls /var

Configuration Hub

The /var directory stands for "**variable**" – it stores files that **change frequently** during system operation.

Dynamic Data Storage

- Holds **runtime data** that changes, such as logs, emails, and caches.
- Unlike /etc (static configs), /var content **grows over time**.

•\

Important Subdirectories

- /var/log/ – **System logs** (e.g., /var/log/messages, /var/log/secure)
- /var/spool/ – Jobs waiting (**cronjobs** included) to be processed (e.g., print or mail queue)
- /var/cache/ – Cached files for applications/package managers
- /var/tmp/ – Temporary files (more persistent than /tmp)
- /var/lib/ – State info for services (e.g., dpkg, rpm, mysql)

Used by Services and Daemons

- Web servers (like Apache) store runtime files here.
- Package managers store metadata in /var.

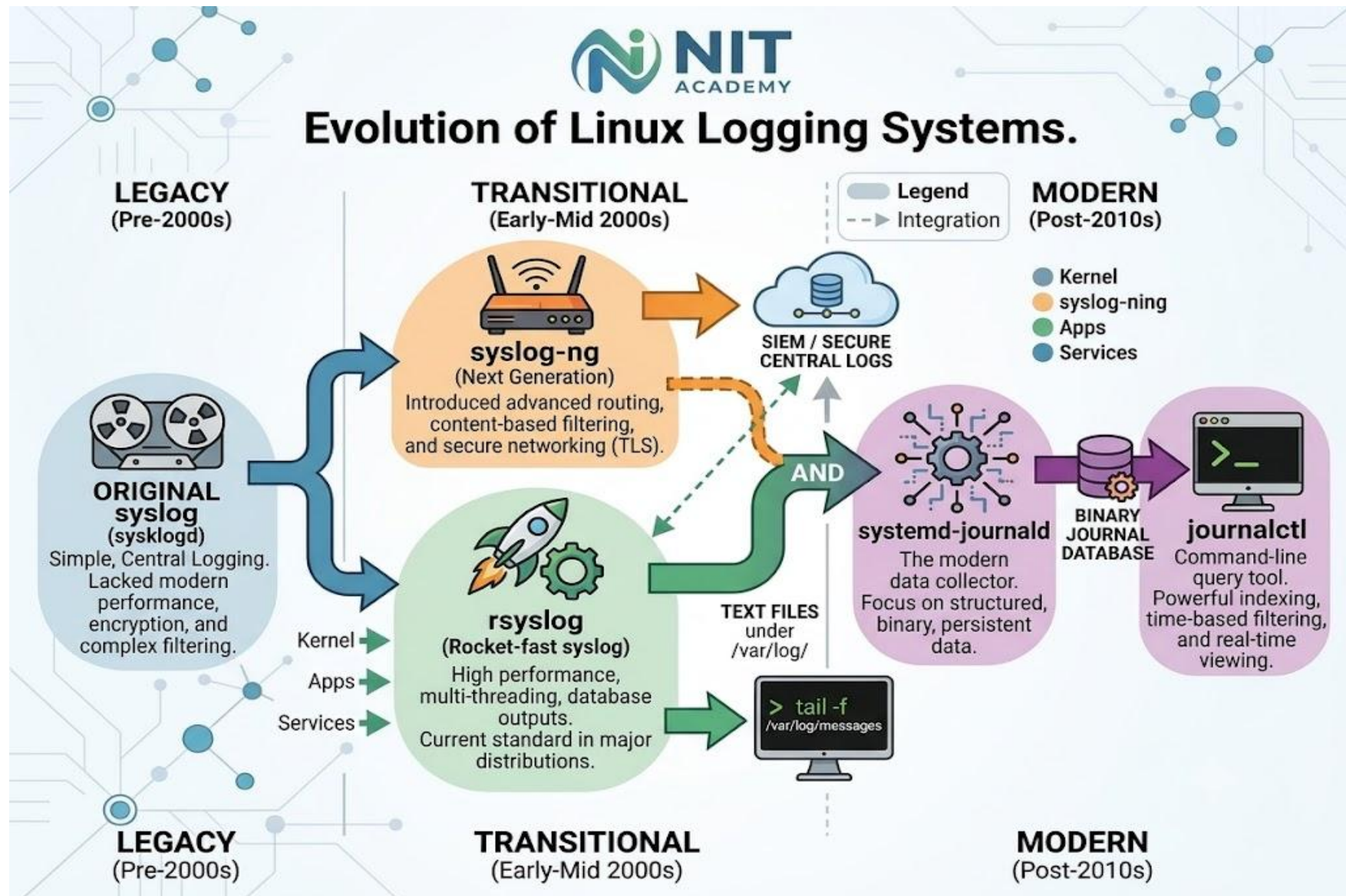
Can Grow Large

- Monitor disk usage in /var – especially /var/log/.
- **Uncontrolled growth can fill up disk and crash services.**

Critical for System Health

- Deleting or mismanaging files here can break logging, updates, or daemons.

Logging – An Overview





How logs work in Linux

The Big Picture

Linux has **two major systems** for logging:

1. **rsyslog** – The **traditional** logging daemon
2. **systemd-journald** – The **newer** logging system used with **journalctl**

Both are involved in **handling logs** — many of which end up in the **/var/log/** directory.

NOTE: "Think of systemd-journald as the **collector**, rsyslog as the **file writer**, and /var/log/ as the **library** where logs live."

1. rsyslog – The Traditional Log Writer

What it does:

Listens for log messages from the kernel, services, and apps.

Where it writes:

Writes logs as **text files** under **/var/log/**, such as:

- /var/log/messages
- /var/log/secure
- /var/log/maillog

Configuration File (remember /etc folder!):

/etc/rsyslog.conf and /etc/rsyslog.d/

Example:

tail -f /var/log/messages

2. journalctl – The systemd Log Viewer

What it does:

Reads logs collected by **systemd-journald**, which captures logs in **binary format** (not plain text).

Where logs are stored:

- /var/log/journal/ → Persistent storage if enabled
- Or in memory (lost after reboot if not configured)

Command to view logs:

journalctl

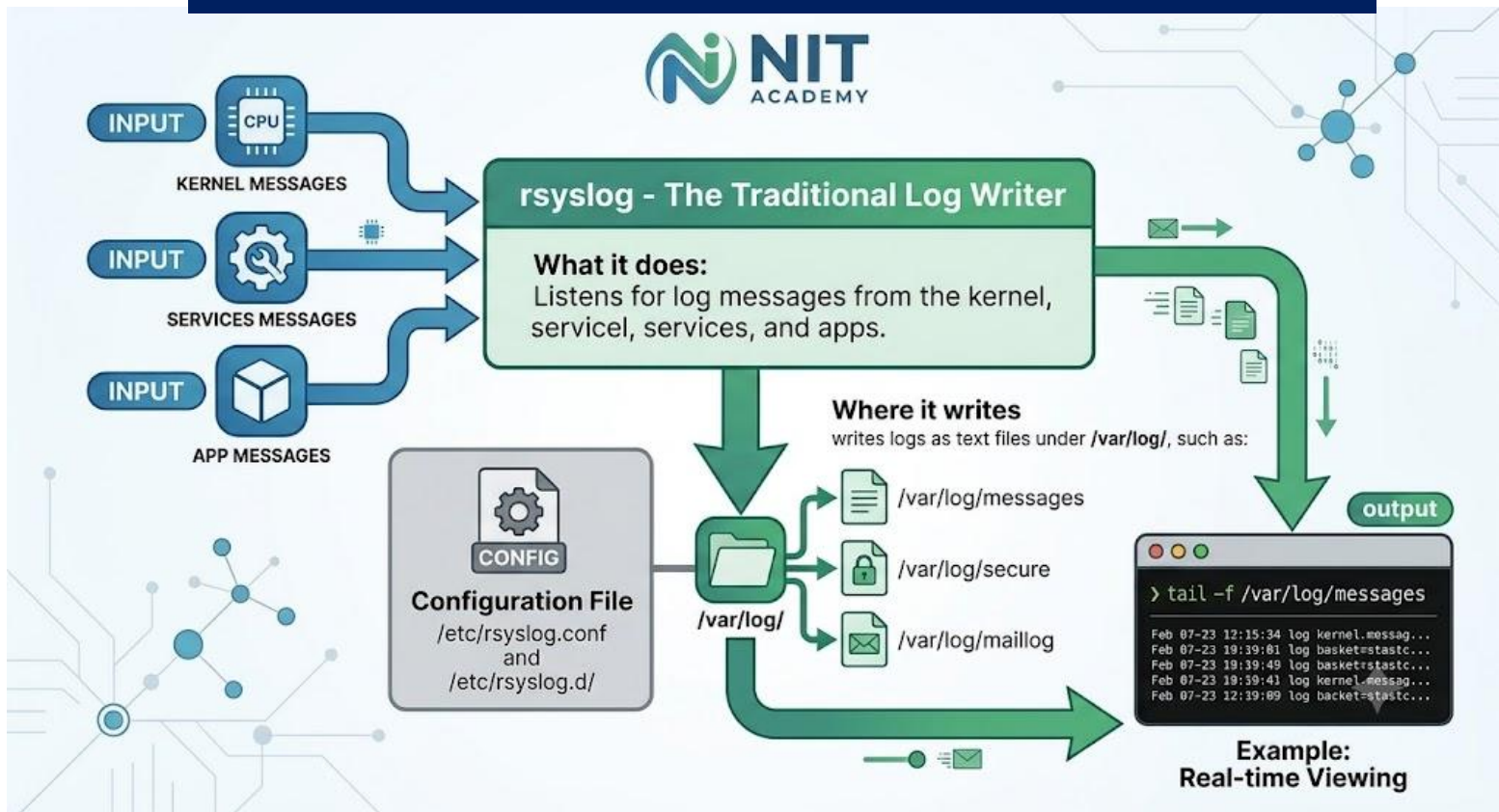
Useful filters:

journalctl -u sshd # Logs for SSH service
journalctl --since "1 hour ago"

How They Work Together

Component	Role	Storage Location
systemd-journald	Collects logs (new system)	/run/log/journal/ or /var/log/journal/
rsyslog	Processes + writes to text files	/var/log/
/var/log/	Directory where traditional logs live	Files created by rsyslog, cron, etc.
journalctl	Reads systemd-journald logs	Uses binary journal data

Rsyslog



rsyslog (The Text Writer)

- **Storage:** It writes plain ASCII text directly to files like `/var/log/messages`.
- **The Benefit:** Anyone and anything can read it instantly with `cat` or `tail`.
- **The Downside:** It is slow to search. If a file has 10 million lines, Linux has to read through line 1, then line 2, then line 3 all the way to the end to find a specific error. There is no index.

EXPLORING RSYSLOG

Where can you find rsyslogs in the filesystem:

[root@practicelab ~]# **ls -l /var/log**

total 15120

```
-rw-----. 1 root root 61266 Jun 27 23:01 cron
-rw-r-----. 1 root root 0 Apr 18 02:50 firewalld
-rw-r--r--. 1 root root 2700 May 30 23:16 hawkey.log-20260531
-rw-rw-r--. 1 root utmp 2336584 Jun 27 23:36 lastlog
-rw-----. 1 root root 0 Jun 21 00:00 maillog
-rw-----. 1 root root 0 May 24 00:00 maillog-20260531
-rw-----. 1 root root 2293198 May 30 23:59 messages
-rw-----. 1 root root 12902 Jun 27 23:36 secure
-rw-----. 1 root root 0 May 24 00:00 spooler-20260531
-drwxr-x---. 2 sssd sssd 87 Jun 21 00:00 sssd
-rw-----. 1 root root 0 Apr 18 02:39 tallylog
-rw-rw-r--. 1 root utmp 116352 Jun 27 23:36 wtmp
```

WATCHING REAL-TIME

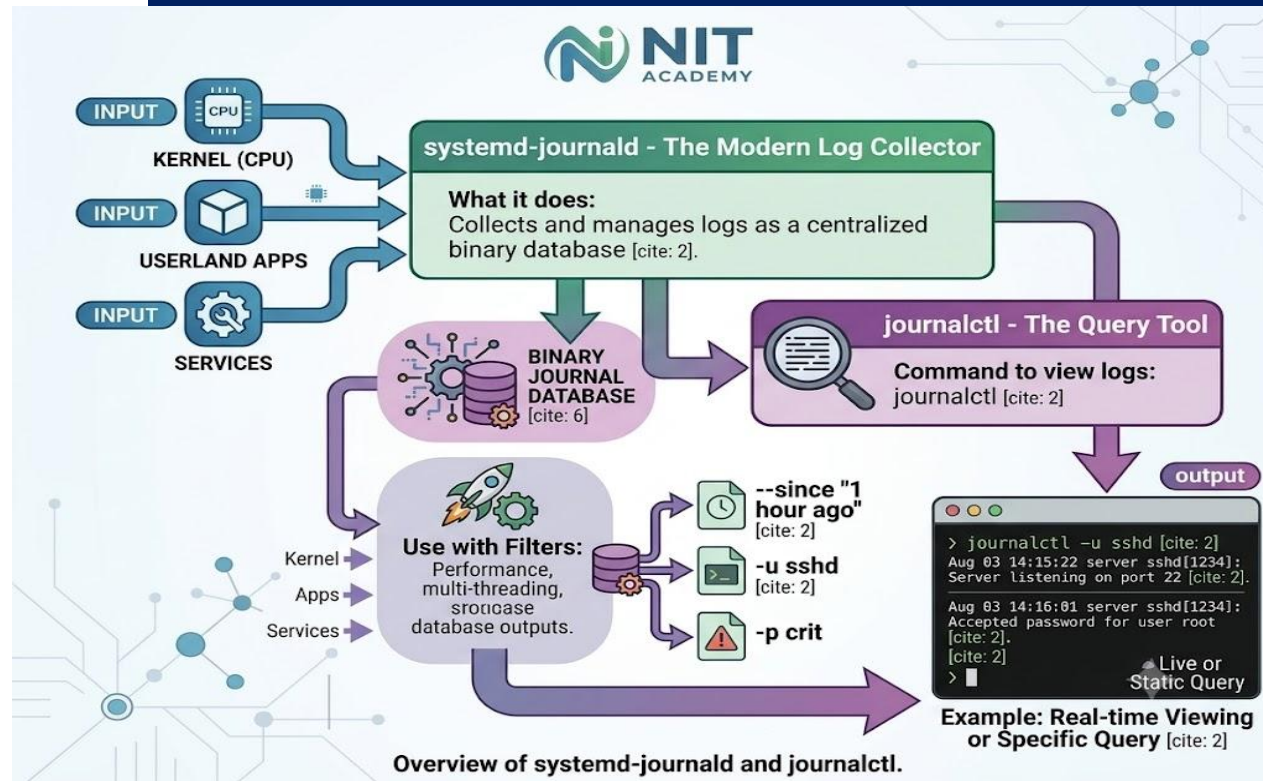
#Run this command in **Terminal 1**:

```
[root@practicelab ~]# tail -f /var/log/messages
```

In **Terminal 2**, use the logger command to inject a manual log message into the system:

```
[root@practicelab ~]# logger "Hello Linux Lab - Testing the log pipeline!"
```

Journalctl – Query Tool



systemd-journal / journalctl (The Database)

- **Storage:** It saves data as a structured, indexed binary database file.
- **The Tool:** journalctl is not a text viewer; it is a database query engine. When you type journalctl, the program instantly queries the binary database, extracts the key-value pairs, and pretty-prints them into human-readable text on your screen.
- **The Benefit:** Because it is a structured database, it supports incredibly fast, complex indexing. You can instantly filter by a specific process ID (**_PID**), a specific system unit (**-u**), or an exact timestamp range (**--since**) without forcing the system to scan millions of lines of text one by one.

EXPLORING JOURNALCTL

Recording events using journalctl

```
[root@practicelab ~]# journalctl --since "1 minute ago"
```

Timestamp (Critical Event Parameter)

```
[root@practicelab ~]# journalctl -n 20
```

system-journald service is stopped

```
[root@practicelab ~]# systemctl status system-journald
```

system-journald service is stopped

```
[root@practicelab ~]# journalctl -u sshd
```

system-journald logs from cloudflared

```
[root@practicelab ~]# journalctl -u cloudflared -n 50 --no-pager
```

WATCHING REAL-TIME

#Run this command in **Terminal 1**:

```
[root@practicelab ~]# journalctl -f
```

In **Terminal 2**, use the logger command to inject a manual log message into the system:

```
[root@practicelab ~]# logger "Hello Linux Lab - Testing the log pipeline!"
```

Journalctl explanation!

We saw the following command. Let us understand it:

```
[root@practicelab ~]# journalctl -u cloudflared -n 50 --no-pager
```

What this means:

- journalctl reads logs from systemd-journald
- Show logs ONLY for the cloudflared service/unit => “-u cloudflared”
- Show logs only the last 50 log lines => “-n 50”
- Print the output directly on the screen instead of opening it inside a scrollable viewer

What is a Pager?

A **pager** is a program that lets you scroll through long output page by page. You have all seen the pager's in this course 😊

Common Pager:

less

SIEM – Security Information and Event Management

SIEM stands for **Security Information and Event Management**.

In the context of logging **infrastructure**, if **systemd-journald** is the local collector and **rsyslog** or **syslog-ng** are the local handlers, then a SIEM is the **central security brain** for the entire enterprise network.

The 4 Core Jobs of a SIEM

1. Centralized Log Aggregation
2. Log Correlation & AI Analysis
3. Real-Time Alerting
4. Compliance and Retention

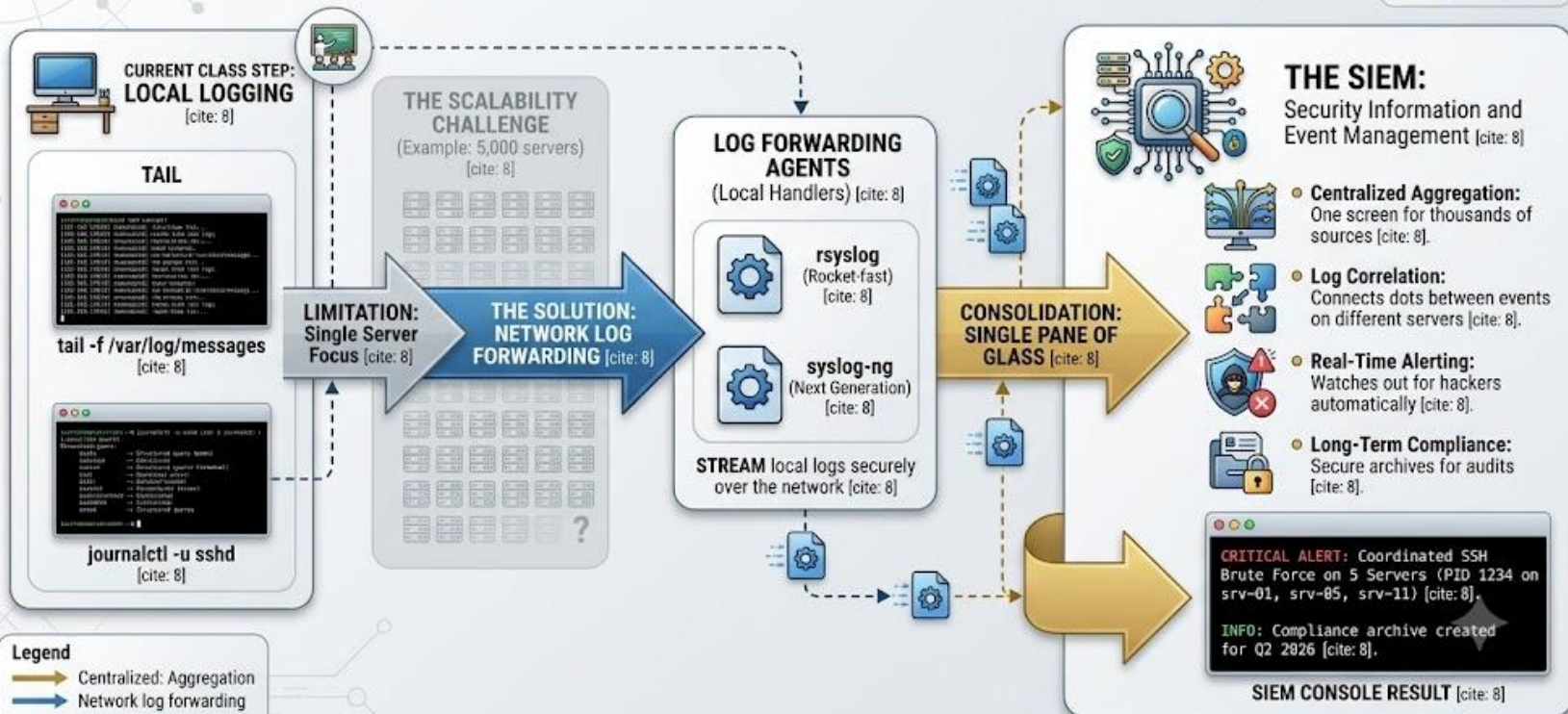
Popular SIEM Tools in the Industry

In corporate IT world, these are the major SIEM tools used:

1. **Splunk**: The heavy hitter of the enterprise world; incredibly powerful data analysis and searching capabilities.
2. **Elastic Security (ELK Stack)**: A massive open-source favorite utilizing Elasticsearch, Logstash, and Kibana to collect and visualize data.
3. **Microsoft Sentinel**: A cloud-native SIEM widely used in hybrid environments.
4. **Wazuh**: A very popular open-source security platform that integrates directly with Linux

SITE RELIABILITY ENGINEERING (SRE)

CONCLUSION: LOCAL LOGS TO ENTERPRISE SECURITY (SIEM) [cite: 8]



Class Date: Jun-2026 [cite: 8]

Logrotate (Linux Utility)

What is logrotate?

Logrotate is a standard utility that Linux uses to keep log files from filling up a hard drive (disk)

logrotate is a **log management tool**. It automatically:

- Rotates (*renames*) logs.
- Compresses old logs.
- Deletes logs older than x-number of days.
- Prevents **/var/log** from filling up.

REMEMBER logrotate relies on two (2) files:

Step 1: Global System Level Configuration File

```
[root@myserver ~]# cat /etc/logrotate.conf
```

- This is the directive / Policy
- What logrotate has to do

Step 2: Service / Application Specific Configuration

```
[root@myserver ~]# ls /etc/logrotate.d/
```

LOGROTATE CONFIG FILE

Global System Level Configuration File – Linux Retention Policy!

[root@practicelab ~]# **cat /etc/logrotate.conf**

rotate log files weekly

weekly

keep 4 weeks worth of backlogs

rotate 4

create new (empty) log files after rotating old ones

create

use date as a suffix of the rotated file

dateext

uncomment this if you want your log files compressed

#compress

packages drop log rotation information into this directory

include /etc/logrotate.d

Logrotate – Retention Policy

Let us understand this **Retention Policy** given in `/etc/logrotate.conf` file:

1. **weekly** defines the *Frequency*

This tells the system **how often** to cut the current active log file and start a fresh one.

Every 7 days:

- the active **app.log** is closed
- The log is **renamed** to something like **app.log-YYYYMMDD** (or **app.log.1**)
- A brand-new **empty app.log** is created.

Therefore, **each archived log file contains exactly 1 week of history.**

2. **rotate 4** defines => *Retention*

This tells the system **how many** of those archived files to keep on the disk before deleting the oldest one.

Retention Policy - Deepdive

Let us understand how this works

When you combine **1-week chunks** with a limit of **4 total files**, you get a rolling 4-week window:

Week 1: app.log.1 (Contains Week 1 data)

Week 2: app.log.1, app.log.2 (Contains Weeks 2 & 1 data)

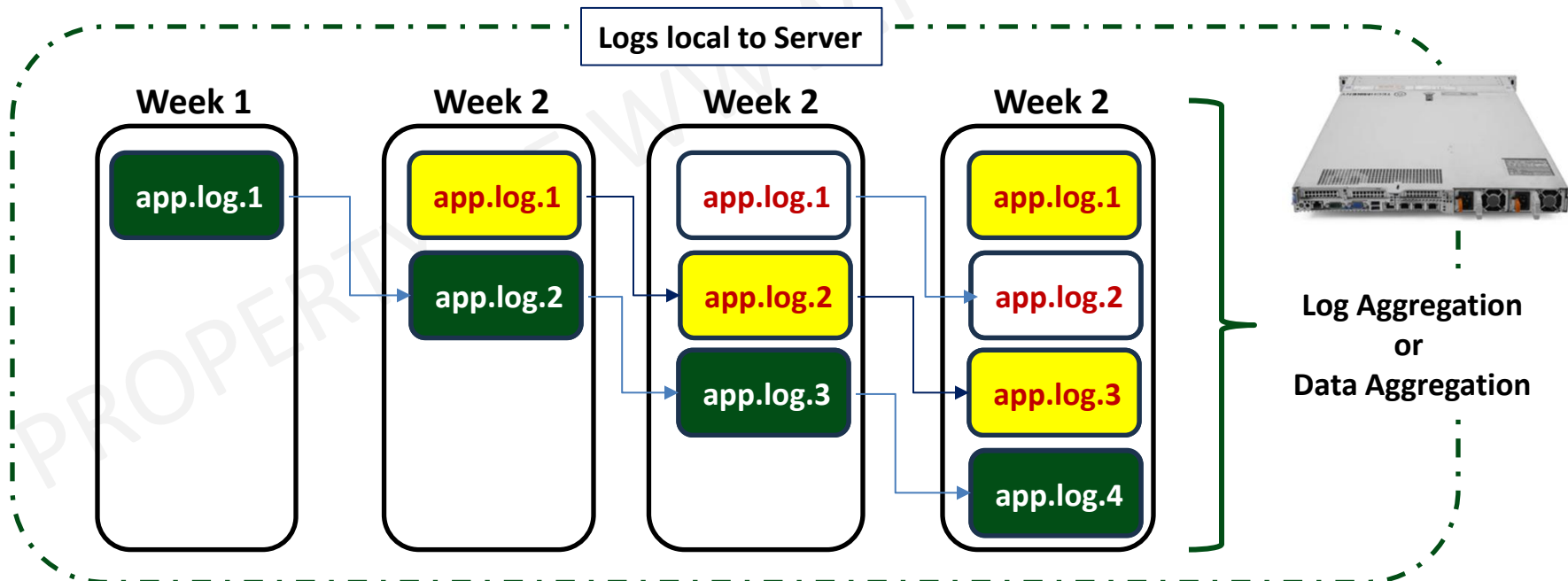
Week 3: app.log.1, app.log.2, app.log.3 (Contains Weeks 3, 2 & 1 data)

Week 4: app.log.1, app.log.2, app.log.3, app.log.4 (Contains Weeks 4, 3, 2 & 1 data — **Full 4-week backlog**)

Week 5: A new rotation happens. **logrotate deletes (rm) the oldest file (app.log.4)** to make room for the new one, keeping your total backlog strictly at the most recent 4 weeks.

Simple Summary: If you changed it to daily and kept rotate 4, you would only have **4 days** of backlogs.

If you changed it to monthly and kept rotate 4, you would have **4 months** of backlogs.



LOGROTATE CONFIG FILE

#Explore Service-Specific Configuration and “cat” rsyslog, dnf and chrony files!

[root@practicelab etc]# **ls -l /etc/logrotate.d/**

total 32

```
-rw-r--r--. 1 root root 130 Oct 14 2019 btmp  
-rw-r--r--. 1 root root 160 Oct 8 2024 chrony  
-rw-r--r--. 1 root root 88 Sep 9 2022 dnf  
-rw-r--r--. 1 root root 93 Mar 11 14:50 firewalld  
-rw-r--r--. 1 root root 162 May 5 12:34 kvm_stat  
-rw-r--r--. 1 root root 226 Nov 4 2025 rsyslog  
-rw-r--r--. 1 root root 289 Nov 11 2025 sssd  
-rw-r--r--. 1 root root 145 Oct 14 2019 wtmp
```


Linux Lab: Understanding Logrotate Compression and Original File Removal

The purpose of this Real-Time Project is to master logrotate.

You will learn how **logrotate**:

1. Rotates an active log file
2. Creates a new empty log file
3. Compresses the old rotated log
4. Removes the uncompressed original after compression
5. Keeps only the compressed archived cop

IMPORTANT:

When talking about logs understand the following:

- Logs are nothing but data.
- System is creating messages (data) based on time (timestamps) and that information is getting stored in the form of a LOG!

Check – Logrotate System Timer!

```
# How did SYSTEMD configure logrotate TIMER?  
[root@practicelab ~]# systemctl list-timers | grep logrotate  
Sun 2026-07-05 00:00:00 CDT 23h left   Sat 2026-07-04 00:00:01 CDT 17min ago  
logrotate.timer           logrotate.service  
# Add sample log data:  
[root@practicelab ~]# cat /var/lib/logrotate/logrotate.status
```

Check – Logrotate STATUS

```
# Understand your  
[root@practicelab ~]# cat /var/lib/logrotate/logrotate.status  
logrotate state -- version 2  
"/var/log/firewalld" 2026-4-19-0:0:0  
"/var/log/hawkey.log" 2026-6-28-0:0:21  
"/var/log/sss/*.log" 2026-4-19-0:0:0  
"/var/log/messages" 2026-6-28-0:0:21  
"/var/log/cron" 2026-6-28-0:0:21
```

Part 1: Create a Practice Log Directory

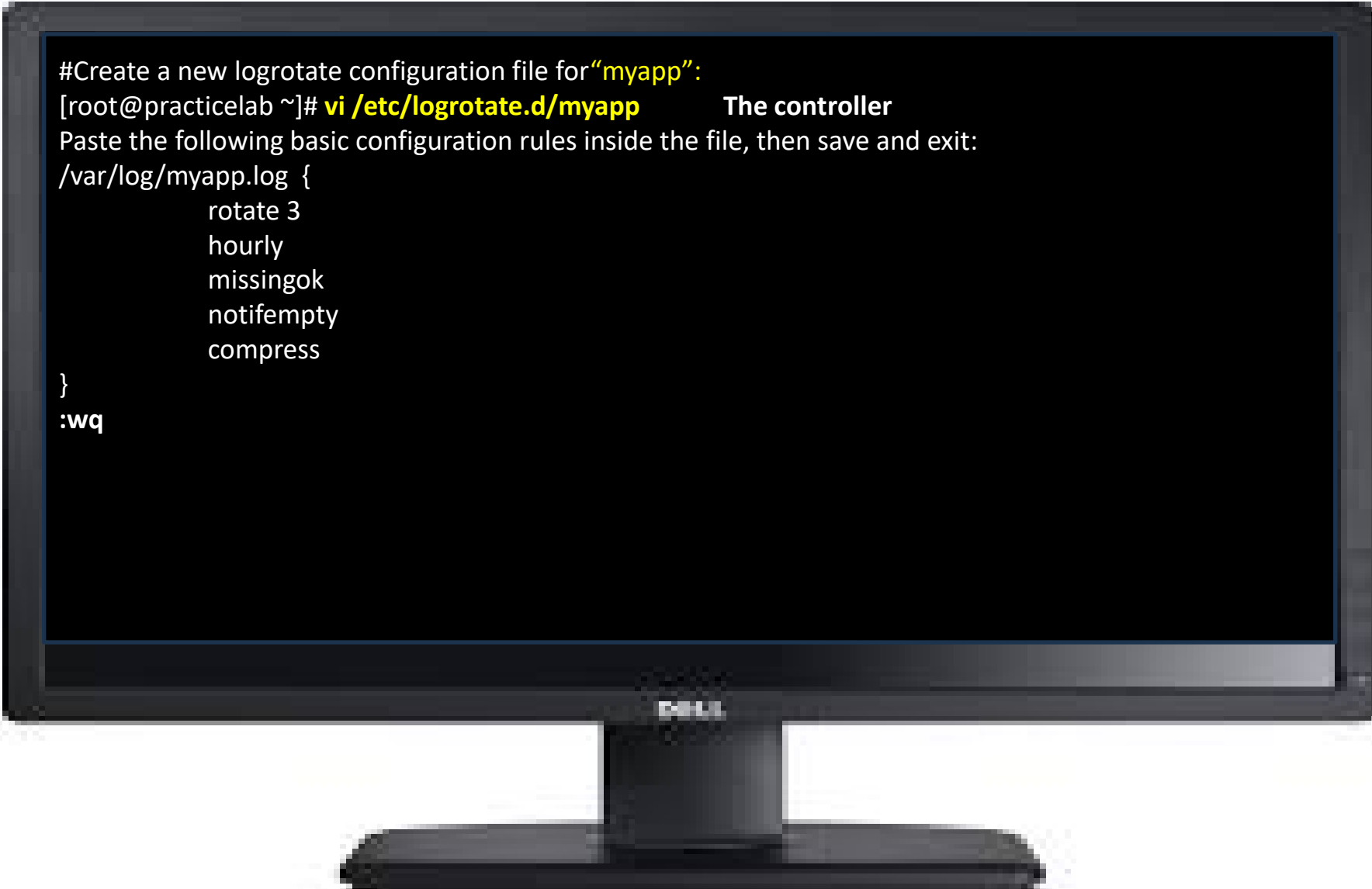
```
# Create a test directory
[root@practicelab ~]# mkdir -p /var/log/myapp.log

# Add sample log data:
[root@practicelab ~]# cat << 'EOF' >> /var/log/myapp.log
User login successful
User opened dashboard
User downloaded report
User logout successful
EOF
```

NOTE: << 'EOF': Tells the shell, "Hey, everything I type next is part of the document, until you see the word EOF on its own line." Putting single quotes around 'EOF' prevents the shell from trying to evaluate variables (like \$USER) inside the text, keeping your log entry exactly as written.

```
# Check and verify that "app.log" has data.
[root@practicelab ~]# cat << 'EOF' >> /var/log/app.log
```

CREATING & ROTATING A DUMMY LOG



```
#Create a new logrotate configuration file for "myapp":  
[root@practicelab ~]# vi /etc/logrotate.d/myapp    The controller  
Paste the following basic configuration rules inside the file, then save and exit:  
/var/log/myapp.log {  
    rotate 3  
    hourly  
    missingok  
    notifempty  
    compress  
}  
:wq
```

CREATING & ROTATING A DUMMY LOG

By default, logrotate runs automatically via a system timer. But all System admins need to run a test and confirm that logs are being rotated. This is done by forcing it right now on **myapp** to see the results immediately

```
[root@practicelab ~]# logrotate -f /etc/logrotate.d/myapp
```

```
[root@practicelab ~]# ls -l /etc/logrotate.d/myapp
```

```
[root@practicelab log]# ls -l /var/log/myapp*
```

```
-rw-r--r--. 1 root root 43 Jun 28 01:54 /var/log/myapp.log.1.gz
```

#Verify the Results:

- The original /var/log/myapp.log is now empty and ready for new data.
- A new file is renamed /var/log/myapp.log.1.gz has appeared.
- This proves that logrotate successfully **rotated** (renamed) and **compressed** the old log file to save disk space!

Summary

Action Item	Target Location	Purpose
Global Settings	<code>/etc/logrotate.conf</code>	Defines default retention and compression behavior.
App Overrides	<code>/etc/logrotate.d/</code>	Houses isolated rules for specific daemons.
Manual Execution	<code>logrotate -f <file></code>	Forces immediate cleanup to prevent disk saturation.

Final Summary

Best Practices

1. Ensure proper permissions (logs often require root access).
2. Regularly rotate logs to prevent large file sizes (managed via **logrotate**).
3. **Logrotate** is used to manage log files, ensuring they don't grow too large or fill up disk space. It can automatically rotate, compress, and delete old logs based on specified rules.
4. Back up critical logs for post-incident analysis.

#cat /etc/logrotate.conf

```
[nitadmin1@localhost ~]$ cat /etc/logrotate.conf
# see "man logrotate" for details
# global options do not affect preceding include directives
# rotate log files weekly
weekly
# keep 4 weeks worth of backlogs
rotate 4
# uncomment this if you want your log files compressed
#compress
# packages drop log rotation information into this directory
include /etc/logrotate.d
# system-specific logs may be also be configured here.
```

Errors and Warnings:

- Look for severity levels, for example, **error**, **critical**, **warn**, **fail**, or **denied**.
- Application logs may use unique severity codes (e.g., Apache logs use **INFO**, **WARNING**, **ERROR**).

#grep "error" /var/log/messages

Time and Date: Each entry has a Timestamp

Source: server or application

Message content: read the content

The big picture – what next

- **logrotate** is completely blind and lifeless on its own.
- **logrotate** is not a background service (daemon). It does not run constantly, and it has no internal clock. It cannot wake itself up at midnight to check if it's "daily" or "weekly". It is just a quick, one-shot script.
- **If nobody triggers logrotate, nothing happens.** The only way to solve this problem is to use to create a trigger.
- **cron** is the alarm clock that wakes logrotate up – the trigger!

How they work together:

1. **The Trigger (cron):** Every day (usually late at night), a system-wide cron job (**/etc/cron.daily/logrotate or a systemd timer**) fires up (triggers) and runs the command => **logrotate /etc/logrotate.conf**.
2. **The Evaluator (logrotate):** logrotate wakes up, looks at user configuration file, checks its internal state file (/var/lib/logrotate/logrotate.status) to see when the files were last rotated, and says:
 1. *"Ah, this file says weekly, and it's only been 3 days. I'll do nothing."*
 2. *"Ah, this file says daily, and it's been 24 hours. Time to rotate it!"*
3. **The Nap:** logrotate finishes its checks and immediately goes back to sleep until cron wakes it up again tomorrow.
4. Without cron, changing your config to daily, weekly, or monthly wouldn't do anything because the logrotate program would never actually execute.

LOGROTATE CONFIG FILE

```
# When cron triggers -> Logscale -> checks the following the timestamps:
[root@practicelab logrotate]# cat /var/lib/logrotate.status
logrotate state -- version 2
"/var/log/firewalld" 2026-4-19-0:0:0
"/var/log/hawkey.log" 2026-6-28-0:0:21
"/var/log/sss/*.log" 2026-4-19-0:0:0
"/var/log/kvm_stat.csv" 2026-4-19-0:0:0
"/var/log/chrony/*.log" 2026-4-19-0:0:0
"/var/log/wtmp" 2026-4-19-0:0:0
"/var/log/spooler" 2026-6-28-0:0:21
"/var/log/btmp" 2026-6-1-0:0:58
"/var/log/maillog" 2026-6-28-0:0:21
"/var/log/sss/sss_kcm.log" 2026-6-29-0:0:14
"/var/log/secure" 2026-6-28-0:0:21
"/var/log/myapp.log" 2026-6-28-14:1:2
"/var/log/messages" 2026-6-28-0:0:21 (linux sees this timestamp and compares to the current system date)
"/var/log/cron" 2026-6-28-0:0:21
```

CRON SCHEDULING WORKFLOW



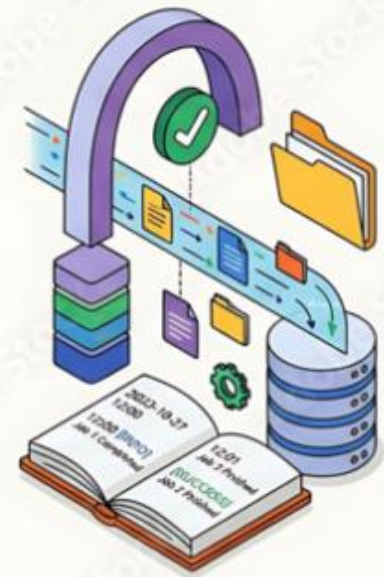
DEFINE SCHEDULES



TRIGGER TASKS



RUN JOBS



LOG ISOLATED BACKGROUND

COPY COMMAND

1. cp (The Standard Copy)

This is used for copying **individual files only**.

What it does: Copies a file from source to destination.

The limitation: If you try to point it at a folder (directory), it will throw an error: cp: folder_name is a directory (not copied).

When to use it: When you are moving a specific file (e.g., cp memo.txt /documents/).

2. cp -r vs cp -R (Recursive Copying)

Both of these flags tell **cp** to copy a folder, including all the files and subfolders inside it. However, there is a hidden difference in how they handle **special files** (like named pipes or device files).

cp -R (The POSIX Standard)

How it works: It copies directories recursively. If it encounters a special file (like a FIFO/named pipe), it recreates that special file at the destination without trying to read its contents.

Why it's better: It is the official POSIX standard. It is safer because it won't get "stuck" trying to read infinite data streams from a special system file.

cp -r (The Traditional/Legacy Way)

How it works: In modern Linux (GNU cp), -r acts exactly the same as -R. However, on older Unix systems or certain BSD variants, -r handles special files differently—it might try to read the content of a special file as if it were a regular text file, which can cause the copy command to hang or crash.

Why to avoid it: It is less portable across different operating systems.

COPY COMMAND

COPY COMMAND PRACTICE LAB SETUP

Create a Directories

```
[root@practicelab ~]# pwd
```

```
/root
```

```
[root@practicelab ~]# mkdir -p cp_lab/dirA/dirB
```

Create Files

```
[root@practicelab ~]# echo "New Data" > cp_lab/file.txt
```

```
[root@practicelab ~]# echo "New Data" > cp_lab/dirA/dirB/file2.txt
```

Get Ready to do the Practicelab

```
[root@practicelab ~]# cd cp_lab
```

```
[root@practicelab cp_lab]# pwd
```

```
/root/cp_lab
```

```
[root@practicelab cp_lab]#
```

“cp” Command

Exercise 1: Use cp for a single file

Goal: Copy a single file to a new location. Run this command:

```
[root@practicelab cp_lab]# pwd
```

```
/root/cp_lab
```

```
[root@practicelab cp_lab]# ls -l
```

```
total 4
```

```
drwxr-xr-x. 3 root root 18 Jul  2 16:19 dirA
```

```
-rw-r--r--. 1 root root 12 Jul  2 16:19 file.txt
```

```
[root@practicelab cp_lab]# cp file1.txt file1_backup.txt
```

```
cp: cannot stat 'file1.txt': No such file or directory
```

NOTE: File name is wrong. To change a file name use move ‘mv’ command

```
[root@practicelab cp_lab]# mv file.txt file1.txt
```

```
[root@practicelab cp_lab]# ls -l
```

```
total 4
```

```
drwxr-xr-x. 3 root root 18 Jul  2 16:19 dirA
```

```
-rw-r--r--. 1 root root 12 Jul  2 16:19 file1.txt
```

```
[root@practicelab cp_lab]# cp file1.txt file1_backup.txt
```

```
[root@practicelab cp_lab]# ll
```

```
total 8
```

```
drwxr-xr-x. 3 root root 18 Jul  2 16:19 dirA
```

```
-rw-r--r--. 1 root root 12 Jul  3 00:49 file1_backup.txt
```

```
-rw-r--r--. 1 root root 12 Jul  2 16:19 file1.txt
```

“cp -R” = “cp -r” Command

Exercise 2: Use cp -R to copy an entire folder structure

Goal: Copy a directory, its subdirectories, and all files inside them.

Run this command:

```
[root@practicelab cp_lab]# cd cp_lab
```

```
[root@practicelab cp_lab]# ll
```

```
total 8
```

```
drwxr-xr-x. 3 root root 18 Jul 2 16:19 dirA
```

```
-rw-r--r--. 1 root root 12 Jul 3 00:49 file1_backup.txt
```

```
-rw-r--r--. 1 root root 12 Jul 2 16:19 file1.txt
```

```
[root@practicelab cp_lab]# cp -R dirA ./destination_dir
```

(Note “.” <- for present directory)

```
[root@practicelab cp_lab]# ll
```

```
total 8
```

```
drwxr-xr-x. 3 root root 18 Jul 3 18:00 destination_dir
```

```
drwxr-xr-x. 3 root root 18 Jul 2 16:19 dirA
```

```
-rw-r--r--. 1 root root 12 Jul 3 00:49 file1_backup.txt
```

```
-rw-r--r--. 1 root root 12 Jul 2 16:19 file1.txt
```


"cp -a" Command

Goal: Copy a file while freezing its exact original "Modified Date" and permissions, rather than updating them to right now.

Difference between standard copying and archive copying.

Run this command:

```
[root@practicelab cp_lab]# cd cp_lab
```

```
[root@practicelab cp_lab]# ll
```

```
total 8
```

```
drwxr-xr-x. 3 root root 18 Jul 2 16:19 dirA
```

```
-rw-r--r--. 1 root root 12 Jul 3 00:49 file1_backup.txt
```

```
-rw-r--r--. 1 root root 12 Jul 2 16:19 file1.txt
```

```
[root@practicelab cp_lab]# cp file1.txt archive_copy1.txt
```

```
[root@practicelab cp_lab]# cp -a file1.txt archive_copy2.txt
```

```
[root@practicelab cp_lab]# ls -l
```

```
total 16
```

```
-rw-r--r--. 1 root root 12 Jul 3 19:00 archive_copy1.txt
```

```
-rw-r--r--. 1 root root 12 Jul 2 16:19 archive_copy2.txt
```

BACKUP & RESTORE

Linux Backup Commands and Restore Command

Backup is the process of copying the data **from source to destination** resource.

- It is used to prevent the loss of data.
- Users can take a backup only of their data.
- The complete backup can only be taken by root.

Linux Commands for Backup

1. [root@practicelab ~]# **dump**
2. [root@practicelab ~]# **scp** (rsync as well)
3. [root@practicelab ~]# **crontab**
4. [root@practicelab ~]# **At job**

To restore the backup created by using the dump command

```
[root@practicelab ~]# restore <options> <destination> <source>
```

Options

- **t** Table of contents
- **r** Restore
- **f** Filename

Good News!

We will learn the Use of **dump & restore** commands when we study **Disk Management**

LOCAL BACKUP

Create a BACKUP Directory

```
[root@practicelab ~]# mkdir -p /BACKUP_LOGS
```

Create a file

```
[root@practicelab ~]# touch student_mappings.txt
```

excellent format for file naming, logging, and scripting

```
[root@practicelab ~]# cp student_mappings.txt student_mappings.txt.bak.$(date +%F_%H%M)
```

```
[root@practicelab ~]# cp student_mappings.txt student_mappings.txt.bak.$(date +%F_%H%M%S)
```

Output Explained – Golden Spec to use in Linux and Bash Scripts

`$(date +%F_%H%M%S)` # Output: `2026-07-02_152958`

`%F` = YYYY-MM-DD (2026-07-02)

`%H` = Hour (15)

`%M` = Minute (29)

`%S` = Seconds (58)

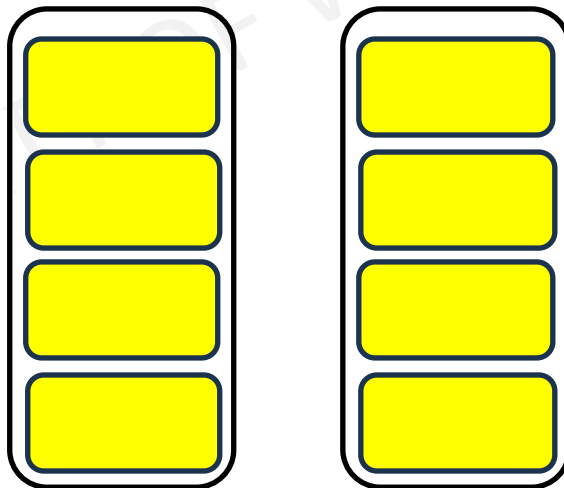
TYPES OF BACKUPS

To see the connection, let's look at what the three types do:

1. **Full Backup:** Copies *everything*. It takes the longest time and uses the most space, but it serves as the ultimate baseline.
2. **Incremental Backup:** Copies only the data that changed *since the last backup of any kind* (yesterday's incremental). It is fast and small.
3. **Differential Backup:** Copies only the data that changed *since the last Full backup*. It grows larger each day until the next Full backup is taken.

Logrotate and Backups

A standard logrotate block doesn't do "incremental" tracking of individual file deltas (it just moves the whole file), it implements a strict **FIFO (First-In, First-Out)** data retention policy that mimics a classic **Grandfather-Father-Son (GFS)** rotation scheme.



DISASTER RECOVERY - DR

To be considered a true disaster recovery solution, a tool must fulfill three fundamental requirements:

1. Offsite / Redundant Storage (The 3-2-1 Rule)

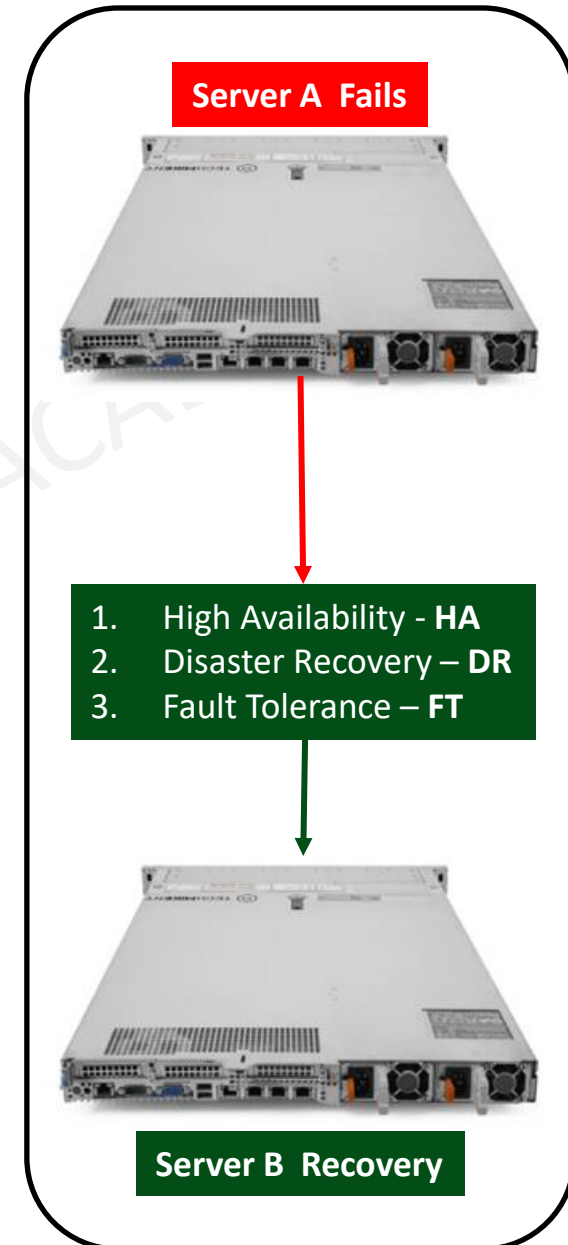
A DR tool does not leave backups on the same disk or even the same machine. It automatically replicates your data to an isolated location. This could be:

- A separate physical server rack or storage array.
- A completely different building or geographic site.
- Secure cloud storage buckets (like AWS S3, Backblaze, etc.).

2. Bare-Metal or Full System Recovery

Standard backups might just save your text files or databases. A DR backup tool can capture the **entire operating system state**:

- including configuration files,
- partition tables,
- user accounts and
- installed applications.



DISASTER RECOVERY - DR

3. Focus on RPO and RTO

DR tools are measured by two critical metrics:

- **Recovery Point Objective (RPO):** How much data can you afford to lose? (e.g., If you back up every 1 hour, your RPO is 1 hour).
- **Recovery Time Objective (RTO):** How fast can you get back online? (e.g., If a server dies, how many minutes or hours does it take to restore the backup and resume production?).

Examples of DR Tools (By Tier)

Depending on where you are running your infrastructure, DR backup tools look a bit different:

Enterprise / Hypervisor Level: Tools like **Veeam** or **Xen Orchestra** (which handles automated VM snapshots, continuous replication, and metadata backups for Type-1 hypervisors).

System / Image Level: Tools like **Clonezilla** (for bare-metal imaging) or **Timeshift / Snapper** (for local OS state snapshots before critical upgrades).

File / Data Level (Offsite): Enterprise-grade synchronization tools like **Restic**, **BorgBackup**, or even highly-customized **rsync** scripts that push encrypted chunks of data over secure tunnels to remote servers.

Examples of DR Tools (By Tier)

Depending on where you are running your infrastructure, DR backup tools look a bit different:

Enterprise / Hypervisor Level: Tools like **Veeam** or **Xen Orchestra** (which handles automated VM snapshots, continuous replication, and metadata backups for Type-1 hypervisors).

System / Image Level: Tools like **Clonezilla** (for bare-metal imaging) or **Timeshift / Snapper** (for local OS state snapshots before critical upgrades).

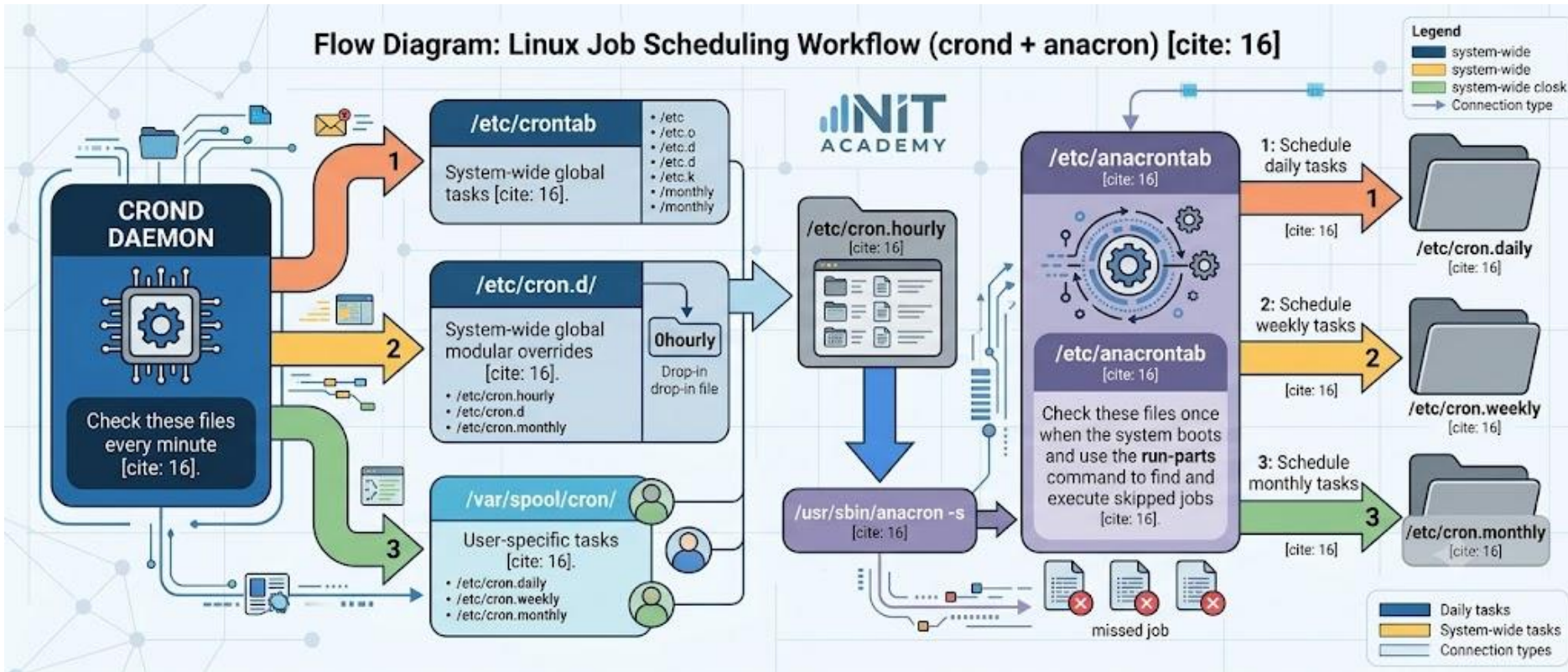
File / Data Level (Offsite): Enterprise-grade synchronization tools like **Restic**, **BorgBackup**, or even

Cronjob – Automation 101



Overview – Cron/Anacron

Flow Diagram: Linux Job Scheduling Workflow (crond + anacron) [cite: 16]



CRON & Anacron

The **cron** is a utility that allows a Linux User to run commands and scripts automatically at a specific interval. It accepts commands and time intervals as arguments. It schedules the specified commands to execute at the provided time intervals.

cron V/s anacron

The **cron** and **anacron** are separate packages but work together to execute scheduled tasks.

- **cron** executes tasks at the scheduled time when the system is up. **If the system is down, the task will not run.**
- **anacron** solves this issue. It **checks all scheduled jobs at boot time and compares their scheduled times** with the system downtime to identify the skipped jobs. If there is any skipped job, it runs that. It works only with daily, weekly, and monthly jobs.

The run-parts command

cron and **anacron** execute the scheduled tasks using **/usr/bin/run-parts** command. The **run-parts** command accepts a directory path as an argument. It executes all scripts, programs, and commands within the given directory in **alphabetic order**. This feature allows a Linux User to run scripts in a pre-defined order. You can prefix the script name with a numeral to arrange scripts in the required order. For example, you have three scripts: **log.sh**, **user.sh**, and **backup.sh**. You want to run these in the following order

0_user.sh

1_log.sh

2_backup.sh

Managing the cron daemon

Managing the crond daemon

The `systemctl` command manages all daemons. It supports many options and arguments. The following table lists the essential options to manage the crond daemon.

NOTE: The crond daemon must be running to use the cron utility.

Command	Description
<code>#systemctl status crond</code>	Show the current status of the crond daemon.
<code>#systemctl start crond</code>	Start the crond daemon in the current session.
<code>#systemctl stop crond</code>	Stop the crond daemon in the current session.
<code>#systemctl enable crond</code>	Enable and start the crond daemon at boot time.
<code>#systemctl disable crond</code>	Disable and start the crond daemon at boot time.

Overview cron – “crontab”

```
[root@practicelab ~]# crontab
```

```
crontab: usage error: file name or - (for stdin) must be specified
```

Usage:

```
crontab [options] file
```

```
crontab [options]
```

```
crontab -n [hostname]
```

Options:

```
-u <user> define user
```

```
-e edit user's crontab
```

```
-l list user's crontab
```

```
-r delete user's crontab
```

```
-i prompt before deleting
```

```
-n <host> set host in cluster to run users' crontabs
```

```
-c get host in cluster to run users' crontabs
```

```
-T <file> test a crontab file syntax
```

```
-s selinux context
```

```
-V print version and exit
```

```
-x <mask> enable debugging
```

/etc/crontab

It is the **main configuration file** of cron. Before anacron, cron used it to execute the **run-parts** command on directories containing daily, weekly, and monthly cron jobs. Later, Linux moved this functionality to anacron. Now, this file contains only environmental variables and a commented crontab syntax that you can use to add entries to this file. Environmental variables let you customize the way you want to execute the jobs. By default, it defines the **SHELL**, **PATH**, **HOME**, and **MAILTO** variables.

```
[root@practicelab ~]# cat /etc/crontab
```

```
SHELL=/bin/bash
```

```
PATH=/sbin:/bin:/usr/sbin:/usr/bin
```

```
MAILTO=root
```

```
# For details see man 4 crontabs
```

```
# Example of job definition:
```

```
# .----- minute (0 - 59)
```

```
# | .----- hour (0 - 23)
```

```
# | | .----- day of month (1 - 31)
```

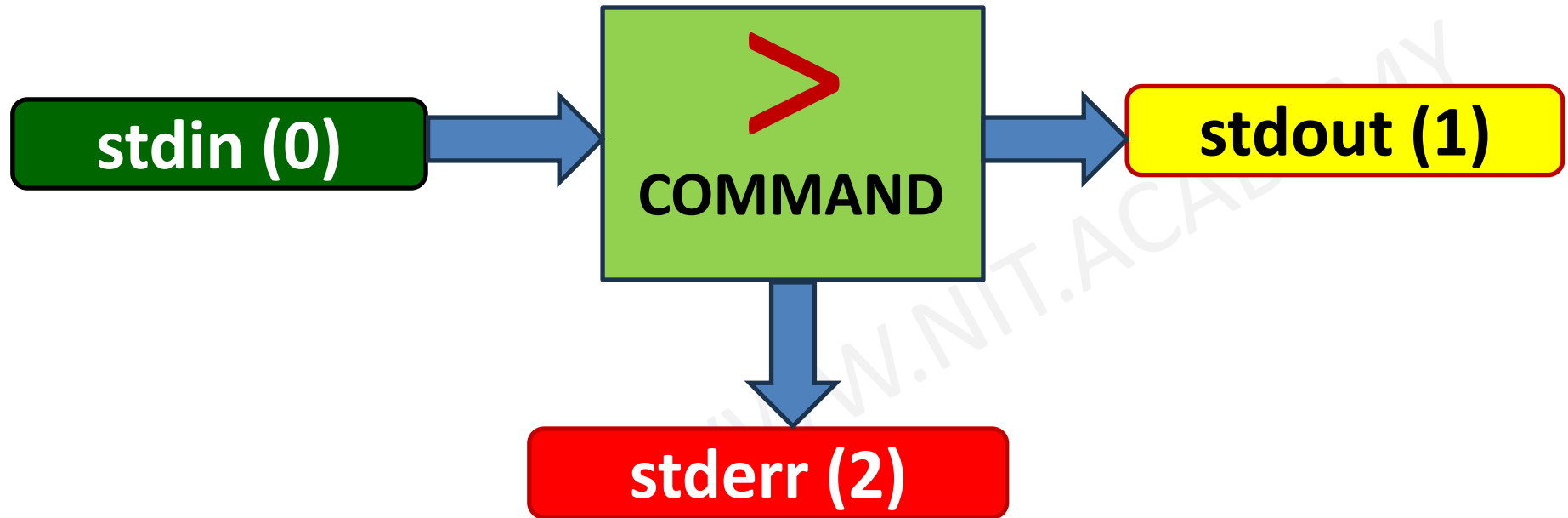
```
# | | | .----- month (1 - 12) OR jan,feb,mar,apr ...
```

```
# | | | | .---- day of week (0 - 6) (Sunday=0 or 7) OR sun,mon,tue,wed,thu,fri,sat
```

```
# | | | | |
```

```
# * * * * * user-name command to be executed
```

REMEMBER THIS?



File Descriptors: Every process opens three standard streams:

0 = Standard Input (stdin)

1 = Standard Output (stdout)

2 = Standard Error (stderr)

SUMMARY REFERENCE TABLE

Operator	What it does	Example
>	Redirects standard output (overwrites file)	ls > files.txt
>>	Redirects standard output (appends to file)	echo "done" >> log.txt
<	Takes input from a file	mysql db_name < backup.sql
2>	Redirects only errors	grep "secret" * 2> errors.txt
2>&1	Merges standard error into standard output (Classical Way)	command > file 2>&1
&>	Merges standard both output and errors (Replaces the classical way of 2>&1)	command &> output.txt
<<	Feeds multi-line text blocks (Here Doc)	cat << EOF
<<<	Feeds a single string (Here String)	bc <<< "5 + 5"

Practice Lab 1 – Simple Cronjob

```
# Check if a job is scheduled (just like ls -l we will use crontab -l)
```

```
[root@practicelab ~]# crontab -l
```

```
[root@practicelab ~]#
```

```
# Let us create a Test cronjob first
```

```
https://tool.crontap.com/
```

```
https://crontab.cronhub.io/
```

```
uptime: /usr/bin/uptime
```

```
[root@practicelab ~]# whereis uptime
```

```
[root@practicelab ~]# crontab -e
```

```
-----
```

```
Press "I" on your key to "Insert" text
```

```
#minute | hour | day_of_month | month | day_of_week
```

```
* * * * * /usr/bin/uptime >> /tmp/uptime.log 2>&1
```

```
press "esc" on your keyboard
```

```
:wq!
```

```
# Let us watch the LIVE logs
```

```
[root@practicelab ~]# tail -f /tmp/uptime.log
```

```
23:22:01 up 3 days, 13:51, 1 user, load average: 0.04, 0.02, 0.00
```

```
23:23:01 up 3 days, 13:52, 1 user, load average: 0.06, 0.03, 0.00
```


Practice Lab 1 – Simple Cronjob

Let us watch the **LIVE** logs

```
[root@practicelab ~]# tail -f /var/log/cron
```

```
Jul 3 23:22:01 practicelab crond[790]: (root) RELOAD (/var/spool/cron/root)
```

```
Jul 3 23:22:01 practicelab CROND[70646]: (root) CMD (/usr/bin/uptime >> /tmp/uptime.log 2>&1)
```

```
Jul 3 23:22:01 practicelab CROND[70644]: (root) CMDEND (/usr/bin/uptime >> /tmp/uptime.log 2>&1)
```

```
Jul 3 23:23:01 practicelab CROND[70662]: (root) CMD (/usr/bin/uptime >> /tmp/uptime.log 2>&1)
```

```
Jul 3 23:23:01 practicelab CROND[70660]: (root) CMDEND (/usr/bin/uptime >> /tmp/uptime.log 2>&1)
```

Press “Ctrl + C” to STOP tailing logs!

Our Test is successful. Let us now LIST the cron job(s) and delete any entries:

```
[root@practicelab ~]# crontab -l
```

```
[root@practicelab ~]# crontab -e
```

Just press “dd” and then “dd” on your keys to delete

```
:wq
```

Practice Lab 2 – Rotate Logs at midnight every Sunday

We want to run a cron job at 12 midnight every Sunday that will move “mv” the logs out of /var/log/ related to app.logs. USE <https://www.uptimia.com/cron-expression-generator> to get the correct answer:

```
[root@practicelab ~]# crontab -l  
[root@practicelab ~]# crontab -e
```

Let us create a Test cronjob first

<https://tool.crontap.com/>

<https://crontab.cronhub.io/>

```
[root@practicelab ~]# crontab -e
```

Press “I” on your key to “Insert” text

#minute | hour | day_of_month | month | day_of_week

* * * * * /usr/bin/uptime >> /tmp/uptime.log 2>&1

press “esc” on your keyboard

:wq!

Let us watch the LIVE logs

```
[root@practicelab ~]# tail -f /tmp/uptime.log
```

23:22:01 up 3 days, 13:51, 1 user, load average: 0.04, 0.02, 0.00

23:23:01 up 3 days, 13:52, 1 user, load average: 0.06, 0.03, 0.00

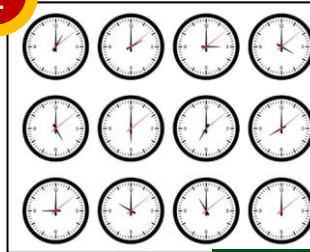
```
# Example of job definition:
# ----- minute (0 - 59)
# ----- hour (0 - 23)
# ----- day of month (1 - 31)
# ----- month (1 - 12) OR jan,feb,mar,apr ...
# ----- day of week (0 - 6) (Sunday=0 or 7) OR sun,mon,tue,wed,thu,fri,sat
# * * * * * user-name command to be executed
```

1



minute

2



Hour

3

OCTOBER

S	M	T	W	T	F	S
			1	2	3	4
5	6	7	8	9	10	11
12	13	14	15	16	17	18
19	20	21	22	23	24	25
26	27	28	29	30	31	

5

S M T W T F S

Day of Week

Day of Month

4

month

2025

JANUARY

S	M	T	W	T	F	S
			1	2	3	4
5	6	7	8	9	10	11
12	13	14	15	16	17	18
19	20	21	22	23	24	25
26	27	28	29	30	31	

FEBRUARY

S	M	T	W	T	F	S
				1	2	3
4	5	6	7	8	9	10
11	12	13	14	15	16	17
18	19	20	21	22	23	24
25	26	27	28	29	30	

MARCH

S	M	T	W	T	F	S
						1
2	3	4	5	6	7	8
9	10	11	12	13	14	15
16	17	18	19	20	21	22
23	24	25	26	27	28	29
30	31					

APRIL

S	M	T	W	T	F	S
			1	2	3	4
5	6	7	8	9	10	11
12	13	14	15	16	17	18
19	20	21	22	23	24	25
26	27	28	29	30		

MAY

S	M	T	W	T	F	S
				1	2	3
4	5	6	7	8	9	10
11	12	13	14	15	16	17
18	19	20	21	22	23	24
25	26	27	28	29	30	31

JUNE

S	M	T	W	T	F	S
						1
2	3	4	5	6	7	8
9	10	11	12	13	14	15
16	17	18	19	20	21	22
23	24	25	26	27	28	29
30						

JULY

S	M	T	W	T	F	S
			1	2	3	4
5	6	7	8	9	10	11
12	13	14	15	16	17	18
19	20	21	22	23	24	25
26	27	28	29	30	31	

AUGUST

S	M	T	W	T	F	S
				1	2	3
4	5	6	7	8	9	10
11	12	13	14	15	16	17
18	19	20	21	22	23	24
25	26	27	28	29	30	31

SEPTEMBER

S	M	T	W	T	F	S
						1
2	3	4	5	6	7	8
9	10	11	12	13	14	15
16	17	18	19	20	21	22
23	24	25	26	27	28	29
30						

OCTOBER

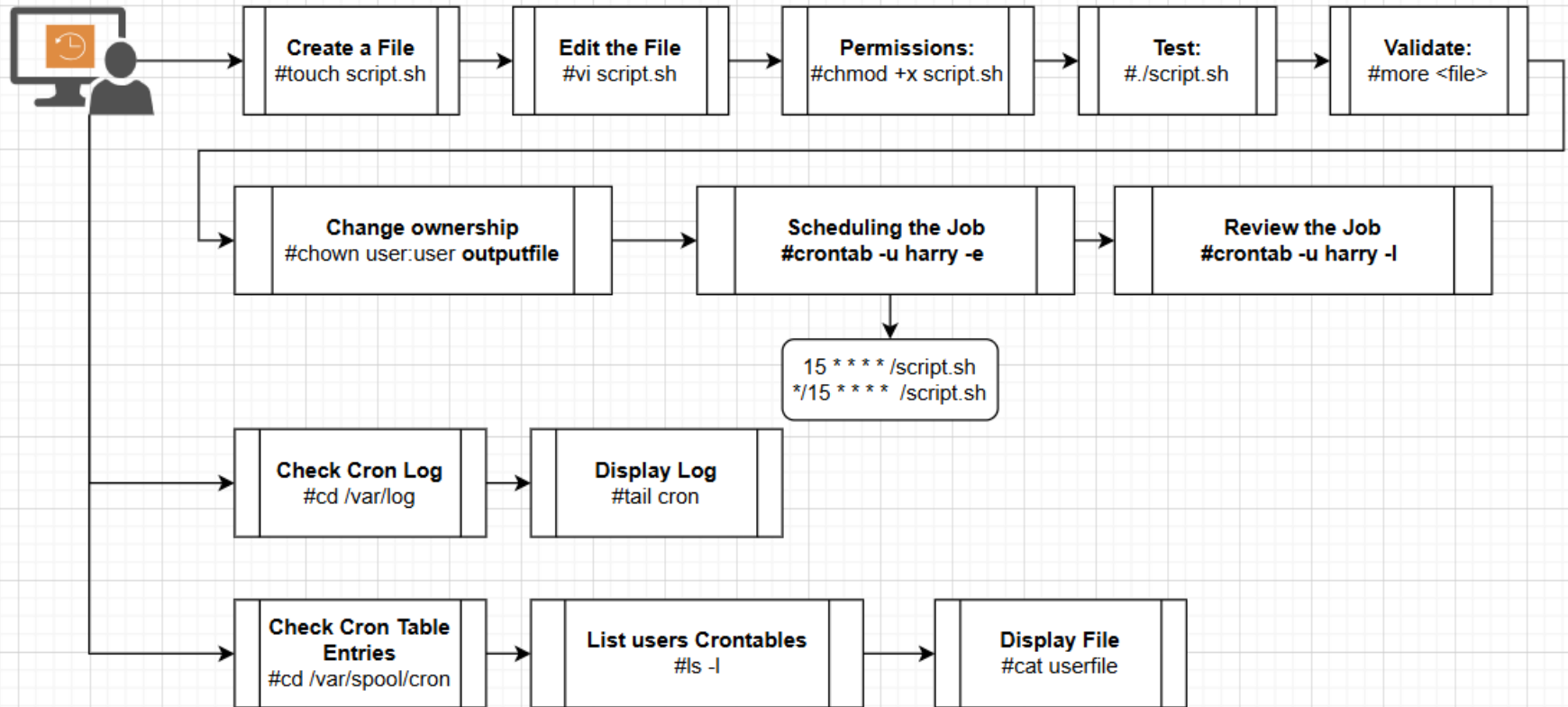
S	M	T	W	T	F	S
			1	2	3	4
5	6	7	8	9	10	11
12	13	14	15	16	17	18
19	20	21	22	23	24	25
26	27	28	29	30	31	

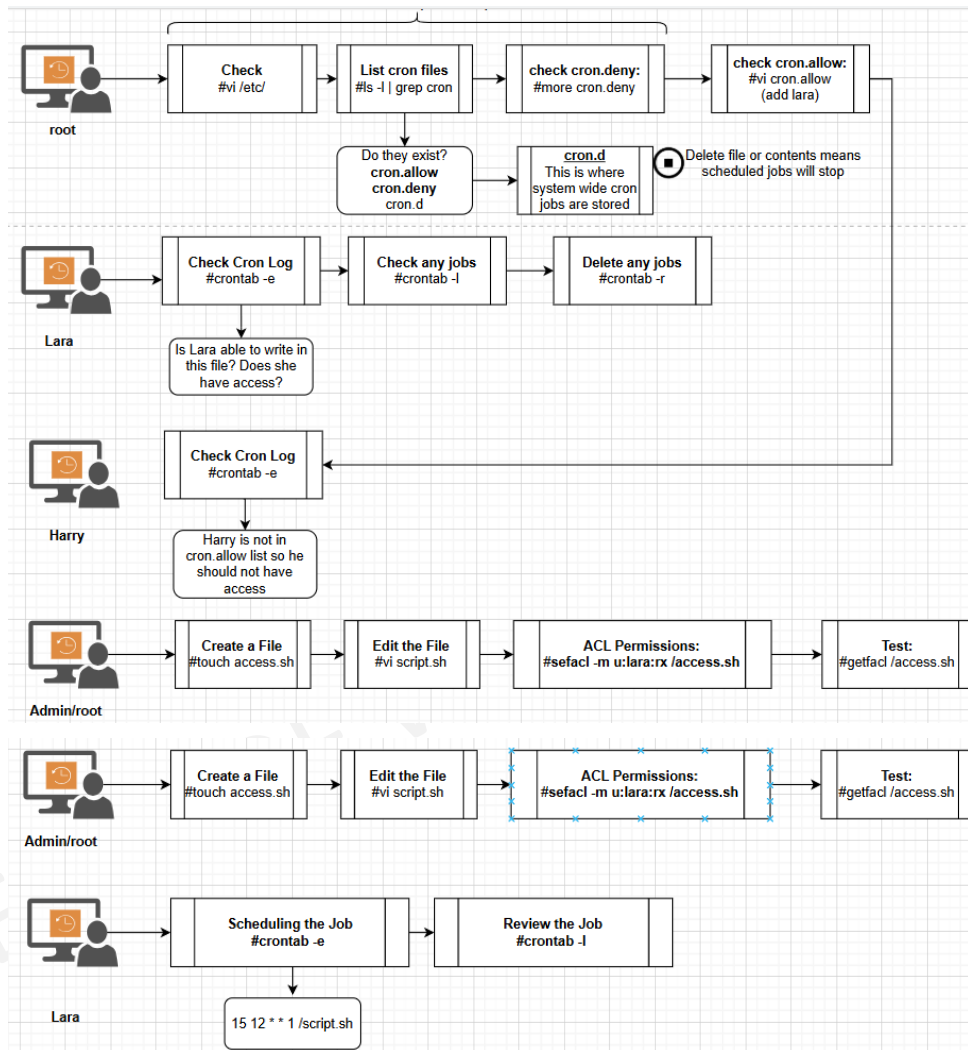
NOVEMBER

S	M	T	W	T	F	S
				1	2	3
4	5	6	7	8	9	10
11	12	13	14	15	16	17
18	19	20	21	22	23	24
25	26	27	28	29	30	

DECEMBER

S	M	T	W	T	F	S
						1
2	3	4	5	6	7	8
9	10	11	12	13	14	15
16	17	18	19	20	21	22
23	24	25	26	27	28	29
30						





/etc/anacrontab

It is the main configuration file of anacron. It contains the configuration that executes the **run-parts** command on directories containing daily, weekly, and monthly cron jobs. It adds a random delay between scheduled tasks to prevent the system from becoming over-exhausted during operations

```
[root@practicelab ~]# cat /etc/anacrontab
# /etc/anacrontab: configuration file for anacron

# See anacron(8) and anacrontab(5) for details.

SHELL=/bin/sh
PATH=/sbin:/bin:/usr/sbin:/usr/bin
MAILTO=root
# the maximal random delay added to the base delay of the jobs
RANDOM_DELAY=45
# the jobs will be started during the following hours only
START_HOURS_RANGE=3-22

#period in days  delay in minutes  job-identifier  command
1    5    cron.daily          nice run-parts /etc/cron.daily
7    25   cron.weekly           nice run-parts /etc/cron.weekly
@monthly 45   cron.monthly          nice run-parts /etc/cron.monthly
```

CRON TO ROTATE LOGS

```
# Create a BACKUP Directory  
[root@practicelab ~]# mkdir -p /BACKUP_LOGS
```

```
# Open the Crontab Editor
```

```
[root@practicelab ~]# crontab -e
```

```
# Add the following Cron Schedule Line
```

```
0 0 * * * mv /var/log/myapp.log.1.gz /BACKUP_LOGS/myapp.log.$(date +%Y%m%d).1.gz
```

Breakdown of the Cron Syntax:

0 0 * * *: Dictates the timing—**Minute 0**, **Hour 0** (12:00 AM Midnight), **Every Day** of the month, **Every Month**, **Every Day** of the week.

mv /var/log/myapp.log.1.gz /BACKUP_LOGS/...: The command that securely transfers the file out of the active logging directory.

\$(date +%Y%m%d): An essential best practice for backups! It **appends** the current date (e.g., 20260628) to the file name. Without this, the cron job would overwrite the previous day's backup file every single midnight.

Note: The backslashes (\) before the % signs are mandatory in cron configurations, as cron treats unescaped % symbols as newlines.

```
# Verify the Cron Job
```

```
[root@practicelab ~]# crontab -l
```



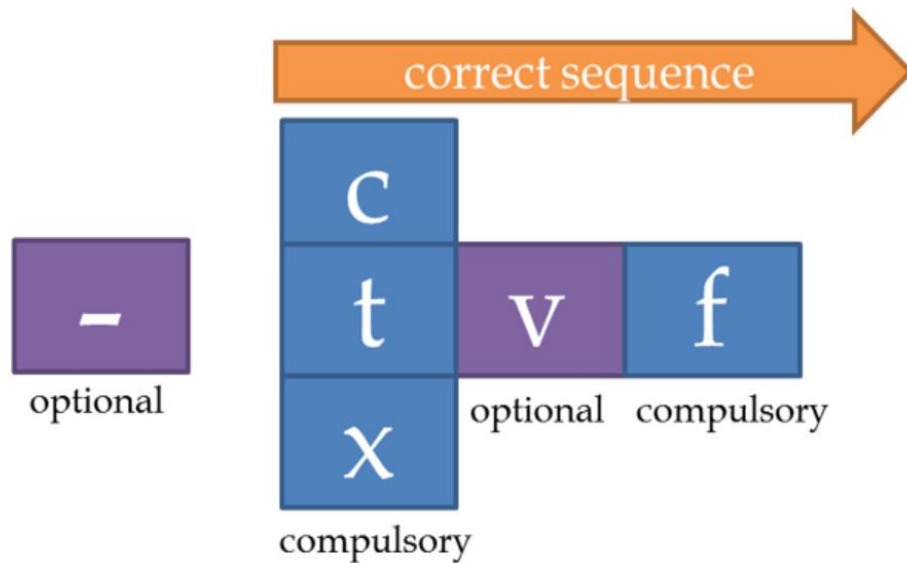
ARCHIVE AND TRANSFER FILES

ARCHIVE AND TRANSFER FILES

TAR

- The **tar** command is one of the most commonly used commands for creating archives.
- Archives allow us to manage several files and directories as a single file.
- Archives are usually created for backup or transferring files

The following image shows the correct sequence of using these options.



The Error: `tar -czfv backup.tar.gz myapp.log`

```
#tar -cvf data.tar app (Create a "TAR" ball)
```

```
#tar -tvf data.tar app (View a "TAR" ball)
```

```
#tar -xvf data.tar app (Extract a "TAR" ball)
```

TAR (COMPRESS)

- The **tar** command does not have any inbuilt functionality to compress files when adding them to the archive. But if required, it can use third-party compression utilities such as the **gzip** and **bzip2** to compress files before adding them to the archive.
- The **gzip** utility compresses faster but provides a low compression ratio.
- The **bzip2** command compresses slower but provides a high compression ratio. In the following section, we will understand how to use both utilities with the **tar** command.

Create a TAR compressed file:

```
#tar -cvf /tmp/home.tar /home  
#tar -czvf /tmp/home.tar.gz /home  
#tar -cjvf /tmp/home.tar.bz2 /home
```

Extract the compressed archived:

```
#tar -xvf data.tar.gz  
#tar -xzvf data.tar.gz
```

Bundling vs Compressing

What is the Key Difference: Bundling vs. Compressing

- **gzip** can only compress **ONE** file at a time. If you pass multiple files to gzip, it will not combine them into a single package. Instead, it will compress each file individually, leaving you with a folder full of separate .gz files.
- **tar is an archiver (a bundler)**. Its only job is to take multiple separate files and folders and glue them together into a single file (called a "tarball"). However, tar by itself does not compress anything—the file size remains exactly the same.

Simple Explanation!

Imagine you are moving to a new apartment and you have 20 loose shirts in your closet. If you use **gzip** directly, it's like putting each individual shirt into its own tiny space-saver vacuum bag. You saved space, but you still have to carry 20 separate bags down the stairs! Using **tar** is like putting all 20 shirts into one big cardboard box so they are easy to carry. Using **tar -z** (tar.gz) is like putting all 20 shirts into that big box, and then vacuum-compressing the entire box so it becomes tiny, light, and easy to move with one hand.

Bundling vs Compressing

Why We Combine Them (tar + gzip)

Imagine your lab generates three different log files for a specific date: myapp.log, secure.log, and nginx.log.

Scenario A: Using only gzip

If you run `gzip myapp.log secure.log nginx.log`, you will end up with:

- myapp.log.gz
- secure.log.gz
- nginx.log.gz

If you need to send these logs to a senior administrator, a developer, or a SIEM team, you have to manage and ship **three separate files**. If you have 50 logs, that becomes a nightmare.

Scenario B: Using tar + gzip (The Professional Way)

By combining them, you get the best of both worlds: **one single file** that is **fully compressed**.

When a user runs `tar -czvf backup.tar.gz /var/log/myapp.log*`, Linux does this in two steps automatically:

1. **tar** gathers all matching log files and binds them together into a single archive stream.
2. **gzip** instantly compresses that entire stream into a single, compact file: **backup.tar.gz**.

PRACTICE LAB

Inspecting the compressed backup directly inside the backup folder!

```
[root@practicelab ~]# zcat /BACKUP_LOGS/myapp.log.*.gz
```

```
[root@practicelab ~]# zless /BACKUP_LOGS/myapp.log.*.gz
```

Extracting a Compressed Log (gunzip)

```
[root@practicelab ~]# cd /BACKUP_LOGS/
```

OPTIONS below:

```
[root@practicelab BACKUP_LOGS]# gunzip myapp.log.1.gz
```

```
[root@practicelab BACKUP_LOGS]# gunzip -k myapp.log.1.gz    (Keeping the original file)
```

```
[root@practicelab BACKUP_LOGS]# gunzip *.gzg
```

NOTE: Running **gunzip** and **gzip -d** is exactly identical!

Verify it is a plain text file again

```
[root@practicelab BACKUP_LOGS]# ls -l
```

Bundling vs Compressing

Cheat Sheet

Memorize the flags:

tar -c = **C**reate a bundle (Pack)

tar -x = **X**tract a bundle (Unpack)

tar -z = Use **Z**ip compression (.gz)

tar -v = **V**iew progress on screen

```
# Syntax: tar -cvf archive_name.tar file_to_archive
```

```
[root@practicelab ~]# tar -cvf backup_review_2026.tar myapp.log.*
```

```
# Create a compressed tarball in one shot
```

```
[root@practicelab ~]# tar -czvf backup_review_2026.tar.gz myapp.log.*
```

```
# Unpack the compressed tarball
```

```
[root@practicelab ~]# tar -xzvf backup_review_2026.tar.gz
```

PRACTICE LAB – MULTIPLE LOGS

```
#Create Multiple Logrotates and send them to BACK-UP
# Inspecting the compressed backup directly inside the backup folder!
[root@practicelab ~]# zcat /BACKUP_LOGS/myapp.log.*.gz
[root@practicelab ~]# zless /BACKUP_LOGS/myapp.log.*.gz

# Extracting a Compressed Log (gunzip)
[root@practicelab ~]# cd /BACKUP_LOGS/
OPTIONS below:
[root@practicelab BACKUP_LOGS]# gunzip myapp.log.1.gz
[root@practicelab BACKUP_LOGS]# gunzip -k myapp.log.1.gz    (Keeping the original file)
[root@practicelab BACKUP_LOGS]# gunzip *.gzg

NOTE: Running gunzip and gzip -d is exactly identical!

# Verify it is a plain text file again
[root@practicelab BACKUP_LOGS]# ls -l
```



REDHAT EXAM QUESTION

NETWORK FILE SHARE (NFS)

Introduction – Network File Sharing (NFS)

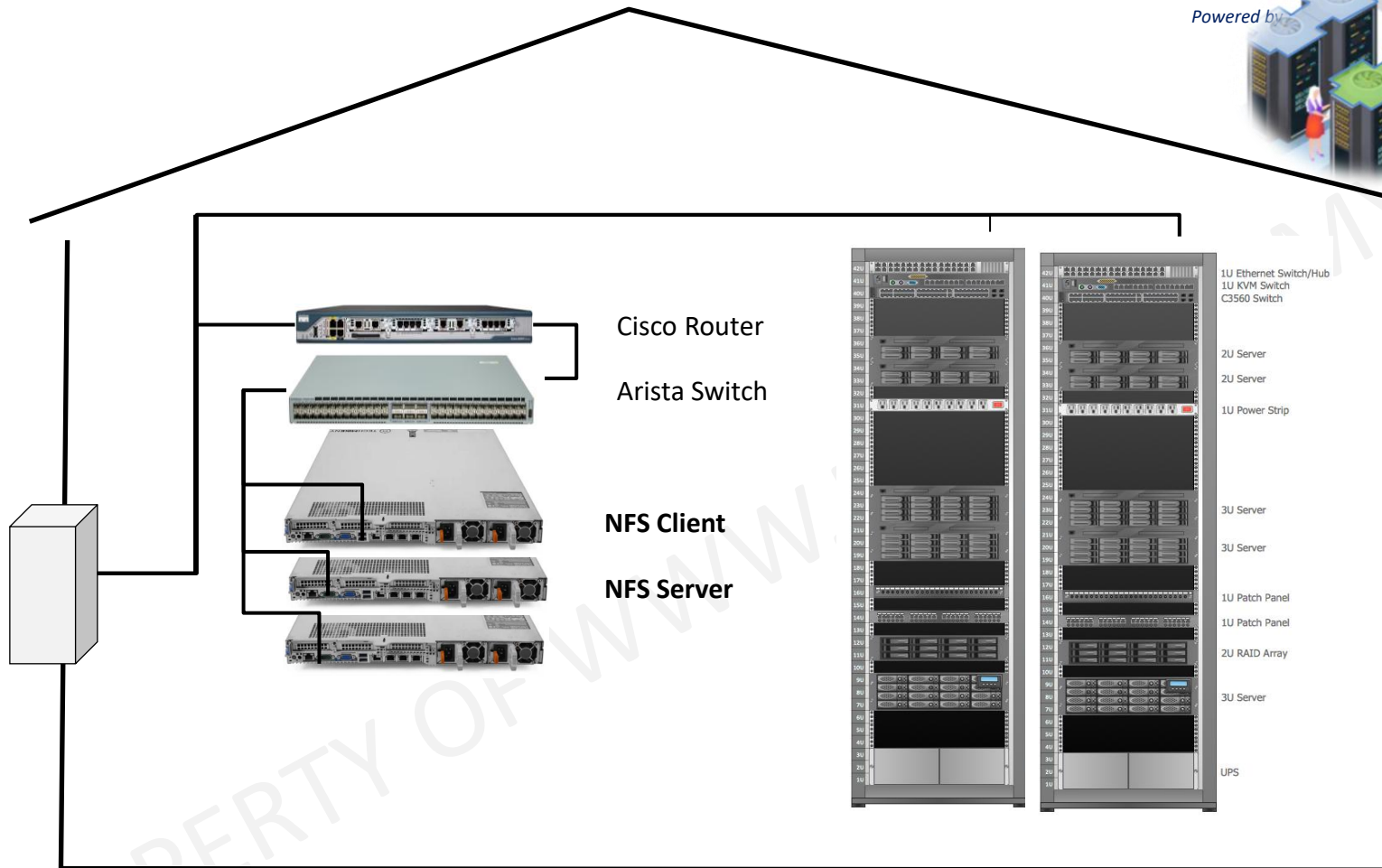
NFS (Network File System) allows users to share local directories on the network and access them from remote systems. It works in a server/client architecture. The nfs server shares directories. The nfs client mounts and uses them as local directories. This tutorial explains how to install, configure, test, and verify it on RHEL/CentOS Linux.

```
# Syntax: tar -cvf archive_name.tar file_to_archive
[root@practicelab ~]# tar -cvf backup_review_2026.tar myapp.log.*

# Create a compressed tarball in one shot
[root@practicelab ~]# tar -czvf backup_review_2026.tar.gz myapp.log.*

# Unpack the compressed tarball
[root@practicelab ~]# tar -xzvf backup_review_2026.tar.gz
```

DATA CENTER



A **server** is a device or **software** that provides **specific services** to clients on a network — such as:

1. File hosting (like an FTP server)
2. Web hosting (like Apache or Nginx)
3. Email services
4. NFS – Network File Sharing
5. DNS or DHCP services (beyond basic routing)

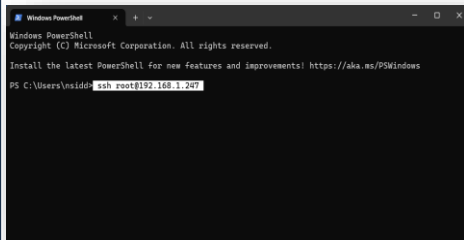
SECURE SHELL (SSH)



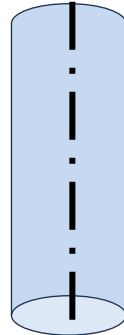
Nexus
Office

SSH

HOME



VPN, Tunnel, OpenVPN



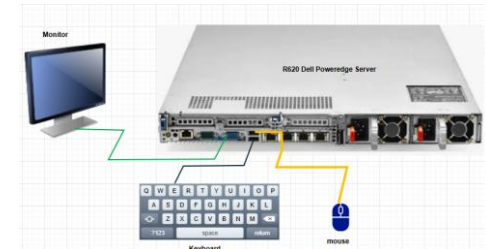
SSH

DATA CENTER

Client

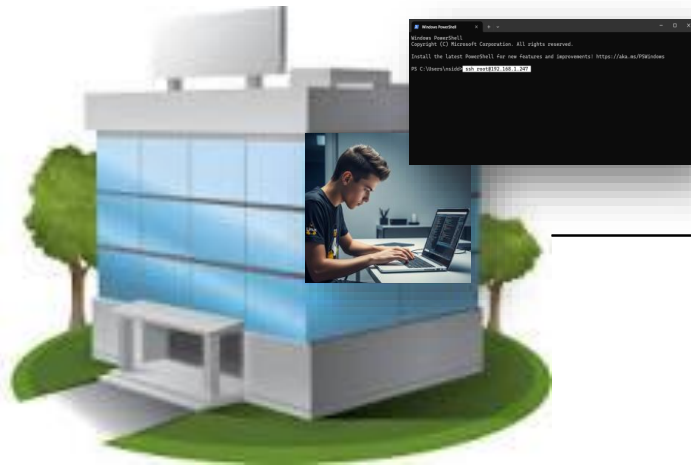


Server



NFS – NETWORK FILE SHARE

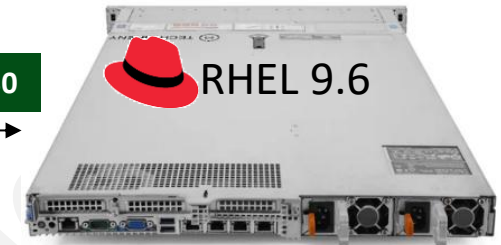
Date: July-26th-2025



SSH

>ssh root@192.168.1.230

NFS Client



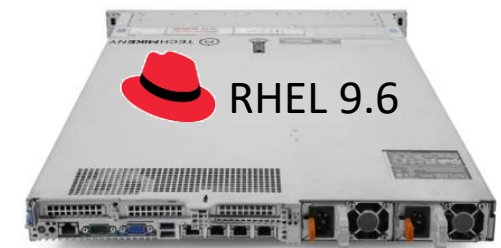
Uses `mount` to access shared folder as if local `/mnt/data`



Over Network via
NFS

>ssh root@192.168.1.231

NFS Server

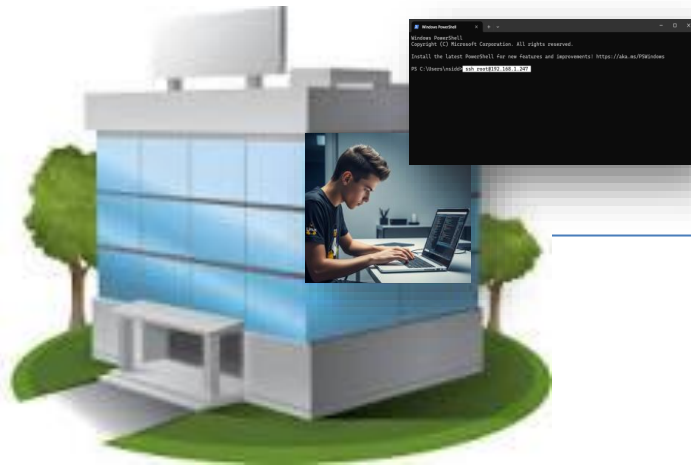


Runs `exportfs` to make folder `/data` available

- We call one a **client** and the other a **server** in NFS terminology this is about the storage relationship:
- The **NFS server** shares a resource.
- The **NFS client** consumes that resource.
- It's about the **NFS file-sharing relationship**, not the general OS type.

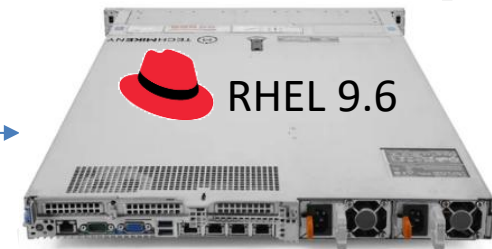
NFS SERVER CONFIGURATION

Date: July-26th-2025



> ssh root@192.168.1.231

NFS Server



Runs `exportfs` to make folder `/data` available

STEP 1: Install NFS Utilities

```
[root@servera ~]# dnf install nfs-utils -y
```

STEP 2: Create Export Directory



CHAGE

Key Password Aging Fields in `/etc/shadow`

- **Last Password Change:** When the password was last changed (in days since the epoch).
- **Minimum Password Age:** The minimum number of days a password must be used before it can be changed.
- **Maximum Password Age:** The maximum number of days a password can be used before it needs to be changed.
- **Password Warning Period:** The number of days before a password expires when the user will be warned.
- **Password Inactivity Period:** The number of days after a password expires that the account will be disabled.
- **Account Expiration:** The date when the account will be disabled (optional).

CHAGE

```
[tom@System-Admin product]$ chage -l tom
Last password change                : Jan 18, 2025
Password expires                    : never
Password inactive                   : never
Account expires                     : never
Minimum number of days between password change : 0
Maximum number of days between password change : 99999
Number of days of warning before password expires : 7
```

Minimum number of days between password change

```
[root@System-Admin tmp]# chage -m 5 tom
```

Maximum number of days between password change

```
[root@System-Admin tmp]# chage -M 90 tom
```

Number of days of **warning** before password expires

```
[root@System-Admin tmp]# chage -w 10 tom
```

Password inactive

```
[root@System-Admin tmp]# chage -I 5 tom
```

Account expires

```
[root@System-Admin tmp]# chage -E 2025-04-01
```

Password expires

```
[root@System-Admin tmp]# chage -d 2025-29-03
```

